

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

ROZŠÍRENIA ELEKTRONICKÉHO PRACOVNÉHO
ZOŠITA PRE LOGIKU – KONTROLÓR
EKVIVALENTNÝCH ÚPRAV
BAKALÁRSKA PRÁCA

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

ROZŠÍRENIA ELEKTRONICKÉHO PRACOVNÉHO
ZOŠITA PRE LOGIKU – KONTROLÓR
EKVIVALENTNÝCH ÚPRAV
BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná Informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra informatiky
Školiteľ: Mgr. Ján Kluka, PhD.

Bratislava, 2026
Jakub Šimon Štofanik



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Jakub Šimon Štofanič
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Kontrola ekvivalentných úprav v interaktívnom logickom pracovnom zošite
Equivalent transformations checking in the interactive logic workbook

Anotácia: V rámci predmetu Logika pre informatikov sa využívajú ekvivalentné úpravy na konverziu formúl do konjunktívneho normálneho tvaru vo výrokovej logike. V spojení so skolemizáciou sa tiež využívajú na vytváranie ekvivalentného klauzálneho tvaru v prvorádovej logike, ktorý sa následne využíva v rezolvenčných dôkazoch. Študentstvo na cvičeniach z predmetu pracuje v elektronickom pracovnom hárku, ktorý vznikol v rámci predchádzajúcej bakalárskej práce (Mok, 2022). Je v ňom síce integrovaný nástroj na kontrolu rezolvenčných dôkazov (Jurík, 2020), kontrola predchádzajúcich ekvivalentných úprav a skolemizácie však chýba. Zadávanie komplexnejších cvičení, kde rezolvenca nadväzuje na takúto ekvivalentné úpravy a skolemizáciu je preto problematické a manuálna kontrola a hodnotenie úprav je veľmi prácna.

Cieľ: Vyvinúť nástroj na kontrolu ekvivalentných úprav a skolemizácie teórií v logike prvého rádu. Integrovať tento nástroj do logického pracovného hárku. Otestovať jeho použitie vo výučbe.

Literatúra: Mok, Matej. Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca. Bratislava : Univerzita Komenského, 2022.
Jurík, Norbert. Webový editor rezolvenčných dôkazov. Bakalárska práca. Bratislava : Univerzita Komenského, 2020.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 08.10.2025

Dátum schválenia: 09.10.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Abstrakt

...

Klíčové slová: ...

Abstract

...

Keywords: ...

Obsah

Úvod	1
1 Teoretické znalosti	2
1.1 Logika prvého rádu a jej syntax	2
1.2 Ekvivalentné a ekvisplnitelné úpravy	6
1.3 Klauzálna teória	8
2 Východiská práce	10
2.1 Prehľad existujúceho ekosystému	10
2.2 Použité technológie	10
2.2.1 HTML, CSS, Bootstrap	10
2.2.2 TypeScript	10
2.2.3 React, React-Bootstrap	11
2.2.4 Redux	11
3 Požiadavky a návrh	12
3.1 Požiadavky	12
3.2 Návrh	13
3.2.1 Návrh stavu aplikácie	13
3.2.2 Návrh kontroly úprav	13
4 Implementácia	16
5 Testovanie	17
Záver	18

Úvod

...uvod...

Kapitola 1

Teoretické znalosti

V tejto kapitole zhrnieme definície potrebných pojmov. Definície sú prebraté z prednášok z Logiky pre informatikov [8]. Budeme používať skratku vtt, ktorá znamená „vtedy a práve vtedy keď“.

1.1 Logika prvého rádu a jej syntax

Logika prvého rádu je rodina jazykov, ktoré umožňujú opisovať vlastnosti a vzťahy objektov z určitej domény pomocou logických spojok a kvantifikácie cez tieto objekty. Jazyky z tejto rodiny zdieľajú logické a pomocné symboly, ale líšia sa v mimologických symboloch. Mimologické symboly opisujú pravé vlastnosti alebo vzťahy medzi objektmi.

Symbolmi jazyka logiky prvého rádu $\mathcal{L}$ sú mimologické, logické a pomocné symboly a individuové premenné. Individuové premenné sú z nejakej nekonečnej spočítateľnej množiny $\mathcal{V}_{\mathcal{L}}$. Mimologické symboly sa delia na funkčné a predikátové symboly a individuové konštanty, ktoré sú v danom poradí z nejakých spočítateľných množín $\mathcal{F}_{\mathcal{L}}$, $\mathcal{P}_{\mathcal{L}}$ a $\mathcal{C}_{\mathcal{L}}$. Logické symboly sa delia na logické spojky (unárna $\neg$ a binárne $\wedge, \vee, \rightarrow$), symbol rovnosti ($\doteq$) a kvantifikátory (existenčný $\exists$ a všeobecný $\forall$). Pomocné symboly sú $(,)$ a $,$ (ľavá zátvorka, pravá zátvorka a čiarka). Množiny $\mathcal{V}_{\mathcal{L}}, \mathcal{C}_{\mathcal{L}}, \mathcal{F}_{\mathcal{L}}, \mathcal{P}_{\mathcal{L}}$ sú vzájomne disjunktné. Logické ani pomocné symboly sa nevyskytujú v symboloch z $\mathcal{V}_{\mathcal{L}}, \mathcal{C}_{\mathcal{L}}, \mathcal{F}_{\mathcal{L}}, \mathcal{P}_{\mathcal{L}}$. Každému symbolu $s \in \mathcal{P}_{\mathcal{L}} \cup \mathcal{F}_{\mathcal{L}}$ je priradená arita $\text{ar}(s) \in \mathbb{N}^+$. Napríklad $\mathcal{V}_{\mathcal{L}_p} = \{x, y, z\}$, $\mathcal{C}_{\mathcal{L}_p} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}_p} = \{\text{father}^1\}$, $\mathcal{P}_{\mathcal{L}_p} = \{\text{cat}^1, \text{loves}^2\}$

Na označenie objektov jazyka slúžia *termy*. Množina $\mathcal{T}_{\mathcal{L}}$ *termov* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platí $\mathcal{V}_{\mathcal{L}} \subseteq \mathcal{T}_{\mathcal{L}}$, $\mathcal{C}_{\mathcal{L}} \subseteq \mathcal{T}_{\mathcal{L}}$ a ak f je funkčný symbol s aritou n a $t_1, \dots, t_n$ patria do $\mathcal{T}_{\mathcal{L}}$, tak aj postupnosť symbolov $f(t_1, \dots, t_n) \in \mathcal{T}_{\mathcal{L}}$. Každý prvok $\mathcal{T}_{\mathcal{L}}$ je *term* jazyka $\mathcal{L}$ a nič iné nie je termom jazyka $\mathcal{L}$. Napríklad $\mathcal{T}_{\mathcal{L}_p} = \{x, y, z, \text{kitty}, \text{father}(x), \text{father}(y), \text{father}(z), \text{father}(\text{kitty}), \text{father}(\text{father}(x)), \text{father}(\text{father}(y))\dots\}$

Rovnostný atóm jazyka $\mathcal{L}$ je každá postupnosť symbolov $t_1 \doteq t_2$, kde t_1 a t_2 sú termy jazyka $\mathcal{L}$. *Predikátový atóm* jazyka $\mathcal{L}$ je každá postupnosť symbolov $P(t_1, \dots, t_n)$, kde P je predikátový symbol s aritou n a $t_1, \dots, t_n$ sú termy jazyka $\mathcal{L}$. *Atomickými formulami* (skrátene *atómami*) jazyka $\mathcal{L}$ súhrnne nazývame všetky rovnostné a predikátové atómy jazyka $\mathcal{L}$. Atómy zodpovedajú jednoduchým výrokom o objektoch jazyka, napríklad „ x je totožný s y “ alebo „ z je mačka“. Množinu všetkých *atómov* jazyka $\mathcal{L}$ označujeme $\mathcal{A}_{\mathcal{L}}$. Napríklad $\mathcal{A}_{\mathcal{L}_p} = \{x \doteq x, x \doteq y, x \doteq \text{father}(x), \dots, \text{cat}(x), \text{cat}(y), \dots, \text{loves}(x, x), \text{loves}(x, z), \text{loves}(x, \text{father}(x)), \dots\}$

Množina $\mathcal{E}_{\mathcal{L}}$ všetkých *formúl* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platí:

1. $\mathcal{A}_{\mathcal{L}} \subseteq \mathcal{E}_{\mathcal{L}}$,
2. Ak A patrí do $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosť symbolov $\neg A$ patrí do $\mathcal{E}_{\mathcal{L}}$ a nazývame ju *negácia* formuly A .
3. Ak A a B sú v $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosti symbolov $(A \wedge B)$, $(A \vee B)$ a $(A \rightarrow B)$ patria do $\mathcal{E}_{\mathcal{L}}$ a nazývame ich postupne *konjunkcia*, *disjunkcia* a *implikácia* formúl A a B .
4. Ak x je individuová premenná a A patrí do $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosti symbolov $\exists x A$ a $\forall x A$ patria do $\mathcal{E}_{\mathcal{L}}$ a nazývame ich postupne *existenčná* a *všeobecná kvantifikácia* formuly A vzhľadom na x .

Každý prvok A množiny $\mathcal{E}_{\mathcal{L}}$ nazývame *formulou* jazyka $\mathcal{L}$.

Nech A je postupnosť symbolov, nech B je formula, nech $Q \in \{\forall, \exists\}$ nech x je premenná. V postupnosti $A = \dots QxB \dots$ sa výskyt formuly QxB nazýva *oblasť platnosti kvantifikátora* Qx v A . Výskyt premennej x v A je *viazaný* vtt sa nachádza v niektorej oblasti platnosti kvantifikátora $\forall x$ alebo $\exists x$ v A . Výskyt premennej x v A je *voľný* vtt sa nenachádza v žiadnej oblasti platnosti kvantifikátora $\forall x$ ani $\exists x$ v A . Premenná x je *viazaná* v A vtt x sa vyskytuje v A a všetky výskyty x v A sú viazané. Premenná x je *voľná* v A vtt x má v A aspoň jeden voľný výskyt. Množinu voľných premenných formuly A označíme $\text{free}(A)$. Pre každú individuovú premennú x , každý symbol konštanty a , každú aritu $n > 0$, každý predikátový symbol P s aritou n , všetky

termy $t_1, \dots, t_n$ a všetky formuly A, B platí:

$$\begin{aligned}
 \text{free}(x) &= \{x\} \\
 \text{free}(a) &= \{\} \\
 \text{free}(t_1 \doteq t_2) &= \text{free}(t_1) \cup \text{free}(t_2) \\
 \text{free}(P(t_1, \dots, t_n)) &= \text{free}(t_1) \cup \dots \cup \text{free}(t_n) \\
 \text{free}(\neg A) &= \text{free}(A) \\
 \text{free}((A \wedge B)) &= \text{free}((A \vee B)) = \text{free}((A \rightarrow B)) = \\
 &= \text{free}(A) \cup \text{free}(B) \\
 \text{free}(\forall x A) &= \text{free}(\exists x A) = \text{free}(A) \setminus \{x\}
 \end{aligned}$$

Formula A jazyka $\mathcal{L}$ je *uzavretá* vtt žiadna premenná nie je voľná v A (teda $\text{free}(A) = \emptyset$). Opak uzavretej formuly je *otvorená* formula. *Uzavreté* formuly zodpovedajú výrokom a *otvorené* formuly zodpovedajú výrokovým formám.

Napríklad pre jazyk $\mathcal{L}_p$ s $\mathcal{C}_{\mathcal{L}_p} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}_p} = \{\text{father}^1\}$, $\mathcal{P}_{\mathcal{L}_p} = \{\text{cat}^1, \text{loves}^2\}$ je

$$X = \exists x \forall y (\text{cat}(y) \rightarrow \text{loves}(x, y)) \wedge \forall x ((\text{loves}(x, \text{kitty})) \rightarrow \text{loves}(\text{father}(\text{kitty}), x))$$

formulou. Nazvime ju X . Neformálne znenie výroku, ktorý zodpovedá X je „Niečo miluje všetky mačky, a zároveň ak niečo miluje kitty, tak ho otec kitty tiež miluje“. Termy použité v X sú y, x , kitty a $\text{father}(\text{kitty})$. Atómy použité v X sú $\text{cat}(y)$, $\text{loves}(x, y)$, $\text{loves}(x, \text{kitty})$ a $\text{loves}(\text{father}(\text{kitty}), x)$. Oblasťou platnosti kvantifikátora $\forall y$ v X je $\forall y (\text{cat}(y) \rightarrow \text{loves}(x, y))$, čo je tiež formula. Nazvime ju Y . X je uzavretá, lebo $\text{free}(X) = \emptyset$ ale Y nie je, lebo $\text{free}(Y) = \{x\}$.

Na priradenie významu mimlogickým symbolom jazyka, sa používa *štruktúra*. *Štruktúrou* pre jazyk $\mathcal{L}$ nazývame každú dvojicu $\mathcal{M} = (D, i)$, kde *doména* D štruktúry $\mathcal{M}$ je ľubovoľná neprázdna množina; *interpretačná funkcia* i štruktúry $\mathcal{M}$ je zobrazenie, ktoré

- každej individuovej konštante c jazyka $\mathcal{L}$ priraduje prvok $i(c) \in D$
- každému funkčnému symbolu f jazyka $\mathcal{L}$ s aritou n priraduje funkciu $i(f) : D^n \rightarrow D$
- každému predikátovému symbolu P jazyka $\mathcal{L}$ s aritou n priraduje množinu $i(P) \subseteq D^n$.

Napríklad $\mathcal{M}_p = (D, i)$ je štruktúra pre $\mathcal{L}_p$, kde $D = \{a, b, c, d, e\}$ a

- $i(\text{kitty}) = d$
- $i(\text{father}) = \{(d, e), (a, b), (b, c), (c, c), (e, e)\}$

- $i(\text{cat}) = \{b, c, d, e\}$
- $i(\text{loves}) = \{(a, b), (b, c), (c, d), (d, e), (e, a), (e, d)\}$

Nech $\mathcal{M} = (D, i)$ je štruktúra pre jazyk $\mathcal{L}$. *Ohodnotenie indivíduových premenných* je ľubovoľná funkcia $e : \mathcal{V}_{\mathcal{L}} \rightarrow D$. Nech ďalej x je indivíduová premenná z $\mathcal{V}_{\mathcal{L}}$ a v je prvok D . Zápis $e(x/v)$ označuje ohodnotenie e' indivíduových premenných, pre ktoré platí $e'(x) = v$ a $e'(y) = e(y)$, ak y je iná premenná ako x .

Nech $\mathcal{M} = (D, i)$ je štruktúra pre jazyk logiky prvého rádu $\mathcal{L}$, nech e je ohodnotenie premenných. *Hodnotou termu* t v štruktúre $\mathcal{M}$ pri ohodnotení premenných e je prvok z D označovaný $t^{\mathcal{M}}[e]$ a zadaný indukčne pre všetky premenné x , konštanty a , každú aritu n , všetky funkčné symboly f s aritou n , a všetky termy $t_1, \dots, t_n$ nasledovne:

$$\begin{aligned} x^{\mathcal{M}}[e] &= e(x) \\ a^{\mathcal{M}}[e] &= i(a) \\ (f(t_1, \dots, t_n))^{\mathcal{M}}[e] &= i(f)(t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]). \end{aligned}$$

Napríklad pre štruktúru $\mathcal{M}_p$ a ohodnotenie $e_p = \{(x, a), (y, b), (z, c)\}$ platí

$$\begin{aligned} x^{\mathcal{M}_p}[e_p] &= e_p(x) = a \\ \text{kitty}^{\mathcal{M}_p}[e_p] &= i(\text{kitty}) = d \\ (\text{father}(\text{kitty}))^{\mathcal{M}_p}[e_p] &= i(\text{father})(\text{kitty}^{\mathcal{M}_p}[e_p]) = e \end{aligned}$$

Nech $\mathcal{M} = (D, i)$ je štruktúra pre $\mathcal{L}$, e je ohodnotenie premenných. Relácia *štruktúra $\mathcal{M}$ spĺňa formulu X pri ohodnotení e* ($\mathcal{M} \models X[e]$) má nasledovnú indukčnú definíciu:

$$\begin{aligned} \mathcal{M} \models t_1 \doteq t_2[e] &\text{ vtt } t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e] \\ \mathcal{M} \models P(t_1, \dots, t_n)[e] &\text{ vtt } (t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]) \in i(P) \\ \mathcal{M} \models \neg A[e] &\text{ vtt } \mathcal{M} \not\models A[e] \\ \mathcal{M} \models (A \wedge B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ a zároveň } \mathcal{M} \models B[e] \\ \mathcal{M} \models (A \vee B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ alebo } \mathcal{M} \models B[e] \\ \mathcal{M} \models (A \rightarrow B)[e] &\text{ vtt } \mathcal{M} \not\models A[e] \text{ alebo } \mathcal{M} \models B[e] \\ \mathcal{M} \models \exists x A[e] &\text{ vtt pre nejaký prvok } m \in D \text{ máme } \mathcal{M} \models A[e(x/m)] \\ \mathcal{M} \models \forall x A[e] &\text{ vtt pre každý prvok } m \in D \text{ máme } \mathcal{M} \models A[e(x/m)] \end{aligned}$$

pre všetky arity $n > 0$, všetky predikátové symboly P s aritou n , všetky termy $t_1, \dots, t_n$, všetky premenné x a všetky formuly A, B jazyka $\mathcal{L}$.

Nech X je uzavretá formula jazyka $\mathcal{L}$ a nech $\mathcal{M}$ je štruktúra pre $\mathcal{L}$. Formula X je *pravdivá* v štruktúre $\mathcal{M}$ (skrátene $\mathcal{M} \models X$) vtt $\mathcal{M}$ spĺňa formulu X pri každom ohodnotení e . Vtedy tiež hovoríme, že $\mathcal{M}$ je *modelom* formuly X . Model množiny formúl T je štruktúra, v ktorej sú pravdivé všetky formuly z T .

Napríklad $\mathcal{M}_p$ je modelom $X = \exists x \text{ loves}(x, \text{kitty})$, pretože: nech e je ľubovoľné ohodnotenie, potom

- $\mathcal{M}_p \models \exists x \text{ loves}(x, \text{kitty})[e]$ vtt
- $\mathcal{M}_p \models \text{loves}(x, \text{kitty})[e(x/a)]$,
- $(x^{\mathcal{M}_p}[e(x/a)], \text{kitty}^{\mathcal{M}_p}[e(x/a)]) \in i(\text{loves})$
- $(a, i(\text{kitty})) \in i(\text{loves})$
- $(a, d) \in i(\text{loves})$

1.2 Ekvivalentné a ekvisplniteľné úpravy

Rozhodnúť o pravdivosti alebo vlastnostiach niektorých formúl je oveľa ťažšie ako iných. Preto je užitočné ich zmeniť do formy, s ktorou sa pracuje ľahšie. Na to slúžia ekvivalentné úpravy. Pomocou nich vieme formulu zmeniť na inú formulu, ktorá má tie isté vlastnosti.

Formula X je *prvorádovo ekvivalentná* s formulou Y ($X \Leftrightarrow Y$) vtt pre každú štruktúru $\mathcal{M}$ a každé ohodnotenie e máme $\mathcal{M} \models X[e]$ vtt $\mathcal{M} \models Y[e]$. Príslovku „prvorádovo“ budeme zvyčajne vynechávať. Množiny formúl S a T sú (prvorádovo) *ekvisplniteľné* (rovnako splniteľné) vtt S má model práve vtedy, keď T má model.

Nech $\mathcal{L}$ je jazyk logiky prvého rádu a nech X, A, B sú formuly jazyka $\mathcal{L}$ také, že $\text{free}(A) \supseteq \text{free}(B)$. *Substitúciou* B za A v X (skrátene $X[A|B]$) nazývame formulu, ktorá vznikne nahradením každého výskytu A v X formulou B . Pre všetky formuly A, B, X, Y jazyka $\mathcal{L}$, všetky individuové premenné x , všetky kvantifikátory $Q \in \{\forall, \exists\}$ a všetky binárne spojky $b \in \{\wedge, \vee, \rightarrow\}$ platí

- $X[A|B] = B$, ak $A = X$
- $X[A|B] = X$, ak X je atóm a $A \neq X$
- $(\neg X)[A|B] = \neg(X[A|B])$, ak $A \neq \neg X$
- $(XbY)[A|B] = ((X[A|B])b(Y[A|B]))$, ak $A \neq (XbY)$
- $(QxX)[A|B] = Qx(X[A|B])$, ak $A \neq (QxX)$

Nech $\mathcal{L}$ je jazyk logiky prvého rádu a nech X je formula, A a B sú výrokovologicky ekvivalentné formuly jazyka $\mathcal{L}$. Potom formuly X a $X[A|B]$ sú tiež výrokovologicky ekvivalentné.

Katalóg ekvivalentných úprav Pri ekvivalentných úpravách sa používa viacero známych dvojíc ekvivalentných formúl. Tieto dvojice vyjadrujú rôzne vlastnosti logických

spojok, ako napríklad komutatívnosť, distributívnosť. Ďalej uvádzame ich zoznam. Pre ľubovoľnú tautológiu $\top$, ľubovoľnú nespĺniteľnú formulu $\perp$, všetky formuly A, B, C , každú premennú x :

$$(A \rightarrow B) \Leftrightarrow (\neg A \vee B) \quad (A \leftrightarrow B) \Leftrightarrow (A \rightarrow B) \wedge (B \rightarrow A) \quad (1.1)$$

$$\neg(A \wedge B) \Leftrightarrow (\neg A \vee \neg B) \quad \neg(A \vee B) \Leftrightarrow (\neg A \wedge \neg B) \quad (1.2)$$

$$\neg\top \Leftrightarrow \perp \quad \neg\perp \Leftrightarrow \top \quad (1.3)$$

$$\neg\neg A \Leftrightarrow A \quad (1.4)$$

$$\neg\exists x A \Leftrightarrow \forall x \neg A \quad \neg\forall x A \Leftrightarrow \exists x \neg A \quad (1.5)$$

$$(A \wedge (B \vee C)) \Leftrightarrow ((A \wedge B) \vee (A \wedge C)) \quad (A \vee (B \wedge C)) \Leftrightarrow ((A \vee B) \wedge (A \vee C)) \quad (1.6)$$

$$(A \wedge (B \wedge C)) \Leftrightarrow ((A \wedge B) \wedge C) \quad (A \vee (B \vee C)) \Leftrightarrow ((A \vee B) \vee C) \quad (1.7)$$

$$(A \wedge B) \Leftrightarrow (B \wedge A) \quad (A \vee B) \Leftrightarrow (B \vee A) \quad (1.8)$$

$$(A \wedge \top) \Leftrightarrow A \quad (A \vee \perp) \Leftrightarrow A \quad (1.9)$$

$$(A \wedge A) \Leftrightarrow A \quad (A \vee A) \Leftrightarrow A \quad (1.10)$$

$$(A \vee (A \wedge B)) \Leftrightarrow A \quad (A \wedge (A \vee B)) \Leftrightarrow A \quad (1.11)$$

$$(A \wedge \perp) \Leftrightarrow \perp \quad (A \vee \top) \Leftrightarrow \top \quad (1.12)$$

$$(A \wedge \neg A) \Leftrightarrow \perp \quad (A \vee \neg A) \Leftrightarrow \top \quad (1.13)$$

$$\exists x(A \vee B) \Leftrightarrow (\exists x A \vee \exists x B) \quad \forall x(A \wedge B) \Leftrightarrow (\forall x A \wedge \forall x B) \quad (1.14)$$

Pre každú formulu D , v ktorej sa x nevyskytuje voľná:

$$\exists x(A \vee D) \Leftrightarrow \exists x A \vee D \quad \forall x(A \vee D) \Leftrightarrow \forall x A \vee D \quad (1.15)$$

$$\exists x(A \wedge D) \Leftrightarrow \exists x A \wedge D \quad \forall x(A \wedge D) \Leftrightarrow \forall x A \wedge D \quad (1.16)$$

$$\exists x D \Leftrightarrow D \quad \forall x D \Leftrightarrow D \quad (1.17)$$

Za podmienky, že y nemá voľný výskyt v A :

$$\exists x A \Leftrightarrow \exists y A\{x \mapsto y\} \quad \forall x A \Leftrightarrow \forall y A\{x \mapsto y\} \quad (1.18)$$

Substitúciou (v jazyku $\mathcal{L}$) nazývame každé zobrazenie $\sigma : V \rightarrow \mathcal{T}_{\mathcal{L}}$ z nejakej množiny individuových premenných $V \subseteq \mathcal{V}_{\mathcal{L}}$ do termov jazyka $\mathcal{L}$. Nech A je postupnosť symbolov (term alebo formula), nech $t, t_1, \dots, t_n$ sú termy a $x, x_1, \dots, x_n$ sú premenné. Term t je *substituovateľný* za premennú x v A vtt nie je pravda, že pre niektorú premennú y vyskytujúcu sa v t platí, že v nejakej oblasti platnosti kvantifikátora $\exists y$ alebo $\forall y$ vo formule A sa premenná x vyskytuje voľná. Substitúcia $\{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je *aplikovateľná* na A vtt term t_i je substituovateľný za x_i v A pre každé $i \in \{1, \dots, n\}$. Nech A je postupnosť symbolov, nech $\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je substitúcia. Ak σ je aplikovateľná na A , tak $A\sigma$ je postupnosť symbolov, ktorá vznikne súčasným nahradením každého voľného výskytu premennej x_i v A termom t_i . Nech $\mathcal{M}$ je štruktúra pre

jazyk $\mathcal{L}$, e ohodnotenie individuových premenných, nech $\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je substitúcia, nech t je term jazyka $\mathcal{L}$ a nech A je formula jazyka $\mathcal{L}$ a σ je aplikovateľná na A . Potom $(t\sigma)^{\mathcal{M}}[e] = t^{\mathcal{M}}[e(x_1/t_1^{\mathcal{M}}[e])\dots(x_n/t_n^{\mathcal{M}}[e])]$ a $\mathcal{M} \models A\sigma[e]$ vtt $\mathcal{M} \models A[e(x_1/t_1^{\mathcal{M}}[e])\dots(x_n/t_n^{\mathcal{M}}[e])]$.

1.3 Klauzálna teória

Jednou metódou dôkazu pravdivosti tvrdenia v logika prvého rádu je rezolvenca. Na riešenie rezolvenacie je potrebný vstup vo forme klauzálnnej teórie. *Literál* je atomická formula $P(t_1, \dots, t_m)$ jazyka $\mathcal{L}$ alebo jej negácia $\neg P(t_1, \dots, t_m)$. *Klauzula* je všeobecný uzáver disjunkcie literálov, teda uzavretá formula jazyka $\mathcal{L}$ v tvare $\forall x_1 \dots \forall x_k (L_1 \vee \dots \vee L_n)$ kde $L_1, \dots, L_n$ sú literály a $x_1, \dots, x_k$ sú všetky voľné premenné formuly $L_1 \vee \dots \vee L_n$. Klauzula môže byť aj jednotková $(\forall x_1 \dots \forall x_k L_1)$ alebo prázdna ($\square$). *Klauzálna teória* je množina klauzúl $\{C_1, \dots, C_n\}$. Môže byť tvorená aj jedinou klauzulou alebo byť prázdna.

Formula X je v *negačnom normálnom tvare* (NNF) vtt neobsahuje implikáciu a pre každú jej podformulu $\neg A$ platí, že A je atomická formula. Formula X je v *prenexnom normálnom tvare* (PNF) vtt má tvar $Q_1 x_1 \dots Q_n x_n A$, kde $Q_i \in \{\forall, \exists\}$, x_i je premenná a A je formula bez kvantifikátorov. Formula X je v *konjunktívnom normálnom tvare* (CNF) vtt má tvar $\forall x_1 \dots \forall x_n A$, kde x_i je premenná a A je konjunkcia disjunkcií literálov.

Skolemizácia je úprava formuly X v NNF, ktorou nahradíme existenčné kvantifikátory novými konštantami alebo funkčnými symbolmi. Ak sa vo formule X vyskytuje $\exists y A$ mimo všetkých oblastí platnosti všeobecných kvantifikátorov, tak pridáme do jazyka novú skolemovskú konštantu c a každý výskyt podformuly $\exists y A$ v X mimo všetkých oblastí platnosti všeobecných kvantifikátorov nahradíme formulou $A\{y \mapsto c\}$. Ak sa vo formule X vyskytuje $\exists y A$ v oblasti platnosti všeobecných kvantifikátorov premenných $x_1, \dots, x_n$ ($X = \dots \forall x_1 (\dots \forall x_2 (\dots \forall x_n (\dots \exists y A \dots) \dots) \dots)$), tak pridáme do jazyka nový funkčný symbol, skolemovskú funkciu f a každý výskyt $\exists y A$ v X v oblasti platnosti kvantifikátorov $\forall x_1, \dots, \forall x_n$ nahradíme formulou $A\{y \mapsto f(x_1, x_2, \dots, x_n)\}$. Pre každú uzavretú formulu X v jazyku $\mathcal{L}$ existuje formula Y vo vhodnom rozšírení $\mathcal{L}'$ jazyka $\mathcal{L}$ taká, že Y neobsahuje existenčné kvantifikátory a X a Y sú ekvivalentné.

Existuje viacero algoritmov *konverzie formuly do klauzálnnej teórie*. My budeme používať nasledujúci algoritmus:

1. Implikácie nahradíme disjunkciami podľa ekvivalentných úprav 1.1.
2. Negačný normálny tvar (NNF): Presunieme negácie k atómom podľa ekvivalentných úprav 1.2 až 1.5.
3. Premenujeme premenné tak, aby každý kvantifikátor viazal inú premennú ako ostatné kvantifikátory podľa ekvivalentných úprav 1.18.

4. Skolemizácia: Existenčné kvantifikátory nahradíme substitúciou nimiviazaných premenných za skolemovské konštanty/aplikácie skolemovských funkcií na príslušné všeobecne kvantifikované premenné.
5. Prenexný normálny tvar (PNF): presunieme všeobecné kvantifikátory na začiatok formuly podľa ekvivalentných úprav 1.14 až 1.17.
6. Konjunktívny normálny tvar (CNF): distribuujeme disjunkcie do konjunkcií.
7. Odstránime konjunkcie rozdelením konjunktov do samostatne kvantifikovaných klauzúl.

Napríklad ekvisplniteľná klauzálna teória pre formulu

$$\exists x \forall y (\text{cat}(y) \rightarrow \text{loves}(x, y)) \wedge \forall x ((\text{cat}(x) \vee \text{loves}(x, \text{kitty})) \rightarrow \text{loves}(\text{kitty}, x))$$

v jazyku $\mathcal{L}$ s $\mathcal{C}_{\mathcal{L}} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}} = \emptyset$, $\mathcal{P}_{\mathcal{L}} = \{\text{cat}^1, \text{loves}^2\}$ dostaneme následovne:

1. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall x (\neg (\text{cat}(x) \vee \text{loves}(x, \text{kitty})) \vee \text{loves}(\text{kitty}, x))$
2. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall x ((\neg \text{cat}(x) \wedge \neg \text{loves}(x, \text{kitty})) \vee \text{loves}(\text{kitty}, x))$
3. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall z ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z))$
4. $\forall y (\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge \forall z ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z))$
5. $\forall y \forall z ((\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z)))$
6. $\forall y \forall z ((\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge ((\neg \text{cat}(z) \vee \text{loves}(\text{kitty}, z)) \wedge (\neg \text{loves}(z, \text{kitty}) \vee \text{loves}(\text{kitty}, z))))$
7. $T = \{\forall y \forall z (\neg \text{cat}(y) \vee \text{loves}(a, y)),$
 $\forall y \forall z (\neg \text{cat}(z) \vee \text{loves}(\text{kitty}, z)),$
 $\forall y \forall z (\neg \text{loves}(z, \text{kitty}) \vee \text{loves}(\text{kitty}, z))\}$

Rozšírený jazyk $\mathcal{L}'$ pre skolemizáciu má $\mathcal{C}_{\mathcal{L}'} = \{\text{kitty}, a\}$, $\mathcal{F}_{\mathcal{L}'} = \emptyset$, $\mathcal{P}_{\mathcal{L}'} = \{\text{cat}^1, \text{loves}^2\}$

Kapitola 2

Východiská práce

2.1 Prehľad existujúceho ekosystému

2.2 Použité technológie

V tejto podkapitole zhrnieme a krátko opíšeme technológie, ktoré sme použili pri vývoji aplikácie.

2.2.1 HTML, CSS, Bootstrap

HTML [3], alebo HyperText Markup Language, je základný jazyk, ktorý sa používa na tvorbu webových stránok. Primárne využitie HTML je na definovanie štruktúry webovej stránky. HTML dokumenty sú tvorené textom a prvkami v stromovej štruktúre. Prvky dokumentu sa označujú začiatočnými a koncovými značkami a môžu mať atribúty. Atribúty prvku riadia ako sa správajú. Na zmenenie vizuálu webových stránok sa používa CSS [2], alebo Cascading Style Sheets. CSS je jazyk, ktorý popisuje ako sa prvky HTML zobrazujú v rôznych médiách. Bootstrap [1] je populárny front-end toolkit, ktorý poskytuje predprívané HTML komponenty, ako sú napríklad tlačidlá, tooltip-y atď. Bootstrap tiež poskytuje CSS štýly.

2.2.2 TypeScript

JavaScript je programovací jazyk, ktorý umožňuje dynamicky meniť obsah webových stránok. TypeScript [7] je jazyk založený na Javascripte. TypeScript obsahuje silné typovanie, čo pomáha pri vývoji ľahším odhalením chýb. Pred spustením aplikácie sa TypeScript prekladá do JavaScriptu, čo umožňuje kompatibilitu s JavaScriptom.

2.2.3 React, React-Bootstrap

React [4] je knižnica pre Javascript, ktorá slúži na vytvorenie používateľského rozhrania webových aplikácií. Základným stavebným kameňom Reactu sú komponenty. Každý komponent je definovaný ako JavaScriptová funkcia, ktorá reprezentuje časť rozhrania. Komponenty sú definované v súboroch s príponou „.jsx“, v ktorých je možné kombinovať Javascript s HTML. Výstup komponentu je JSX element. Komponenty môžu mať stav, ktorý React kontroluje. Ak sa stav komponentu zmení, React aktualizuje iba príslušnú časť rozhrania. Na využitie stavu React používa takzvané *hooky*. React-Bootstrap [5] je knižnica, ktorá prepája React s Bootstrapom. Umožňuje vytvárať Bootstrapove komponenty v tvare Reactových komponentov.

2.2.4 Redux

Redux [6] je knižnica v jazyku JavaScript určená na spravovanie globálneho stavu v aplikácii. Redux používa nemenný *store* na ukladanie globálneho stavu. Na zmenu stavu používajú *reducery* a *akcie*. Reducer je funkcia, ktorá na vstupe dostane stav a akciu. Keďže store je nemenný, reducer vráti nový stav zmenený podľa akcie. Akcia je objekt, ktorý popisuje identifikátor zmeny stavu, a prípadne obsahuje ďalšie voliteľné parametre. Na získanie aktuálneho stavu Redux používa *selektory*. Selektor je funkcia, ktorá na vstupe dostane stav a vráti informáciu získanú zo stavu. Redux sa často používa s Reactom na manažovanie stavu Reactových komponentov. Reactové komponent môžu používať hooky pre použité selektorov a zmenu stavu.

Kapitola 3

Požiadavky a návrh

V tejto kapitole zhrnieme požiadavky aplikácie a opíšeme jej návrh. V návrhu opíšeme internú štruktúru aplikácie a na vyššej úrovni opíšeme kontrolu ekvivalentných úprav.

3.1 Požiadavky

Požiadavky vyplývajú z toho, že celkovým cieľom aplikácie je kontrola ekvivalentných úprav. Prvou požiadavkou aplikácie je, že používateľ musí byť schopný pridávať osobitné formuly. Aplikácia musí byť schopná kontrolovať či formuly sú správne napísané, prípadne oznámiť používateľovi ak urobil chybu. Aby bolo možná kontrola správneho napísanie formuly aplikácia potrebuje jazyk, v ktorom sú formuly napísané. Preto druhou požiadavkou je aby bolo možné zadať aplikácii mimologické symboly jazyka formúl.

Hlavnou požiadavkou je samotná kontrola úprav. Kontrolovanie viacerých rôznych úprav by mohlo viesť k tomu že študent nepochopí úplne tomu čo robí, preto aplikácia bude kontrolovať iba jeden typ úpravy v jednom kroku. Používateľ vyberie akú úpravu má pre dva dané formuly kontrolovať. Počas vývoja sa ale ukázalo, že aby bol kontrolór pohodlnejší na používanie je potrebné kontrolovať aj sofistikovanejšie úpravy, ako je napríklad asociativita alebo komutativita. Hlavné použitie aplikácie je pre kontrolu prevodu formuly do CNF tvaru. Aplikácia by preto mala pracovať s postupnosťami ekvivalentných úprav a mala byť schopná pracovať s viacerými postupnosťami.

Aplikácia je určená pre výučbu predmetu Logika pre informatikov, kde sa hlavne používa elektronický pracovný zošit, vyrobený ako súčasť ???. Väčšina úloh pre študentov predmetu je zadávaná a riešená v elektronickom zošite, preto požiadavkou je, že aplikácia by mala byť do neho integrovaná. Táto požiadavka znamená, že používateľské rozhranie bude vyrobené pomocou knižnice React a tiež že bude možné stav aplikácie ľahko uložiť a načítať.

3.2 Návrh

3.2.1 Návrh stavu aplikácie

Stav aplikácie bude rozdelený na 3 časti: jazyk, úpravy a ukladanie/načítanie. Každá časť bude uchovávať iba nevyhnutné dáta. V stave nebude potrebné ukladať komplexnejšie dáta, ktoré vieme odvodiť z uložených.

Prvá časť stavu bude určená pre logický jazyk, konkrétne pre mimologické symboly. Jazyková časť si bude pamätať text vložený do textových polí pre konštanty, predikáty a funkcie. Ale všetky formuly využívajú logický jazyk pri premene do stromových reprezentácií, čo znamená ak sa hodnoty budú často meniť, tak sa budú všetky formuly a úpravy často prepočítavať. Preto si jazyková časť bude tiež pamätať posledné validné vstupy pre texty, ktoré sa použijú pri kontrolovaní formúl.

Druhá časť stavu bude pre samotné formuly a úpravy. Časť si bude pre každú formulu pamätať jej jedinečné identifikačné číslo, text z textového polia, zvolenú ekvivalentnú úpravu a číslo predošlej formuly (formuly, na ktorú sa použila úprava). Keďže s úpravami patria do postupností, v stave budú tiež zapamätané postupnosti identifikačných čísel a tiež identifikačné číslo danej postupnosti. Pre podporu skolemizácie bude v tejto časti vstupy textových polí pre skolémové konštanty a funkcie a identifikačné číslo formuly, pri ktorej vznikli.

Posledná časť bude určená pre ukladanie a načítanie stavu. Jediné čo bude v tejto časti zapísané je chybová hláška ak načítanie stavu zlyhá.

3.2.2 Návrh kontroly úprav

Hlavný problém pri kontrole ekvivalentných úprav je, že úprava mohla nastať ľubovoľný počet krát v ľubovoľnej časti formuly. Ak by sme kontrolovali každé miesto kde by mohla nastať úprava, riešenie by bolo príliš neefektívne a pomalé. Ale môžeme využiť to, že jedna formula vznikla úpravou druhej formuly, a preto buď je časť prvej formuly rovnaká s časťou druhej formuly, alebo nastala úprava. Ak pri súčasnom rekurzívnom prechádzaní stromových reprezentácií oboch formúl nájdeme rozdiel, môžeme usúdiť že sa používateľ na danom mieste pokúsil urobiť ekvivalentnú úpravu. Problém nastane keď niektoré ekvivalentné úpravy nemenia časti formúl, ako napríklad formuly 1.8, kde obidve formuly zdieľajú tú istú spojku. Týmto spôsobom by sme preskočili rovnakú spojku a našli rozdielne neekvivalentné podformuly, z toho by sme zle usúdili, že nastala chyba. Toto vyriešime tým, že chyby budú „prebublávať“ hore. Ak sa program rekurzívne dostal do koreňa A a našiel chybu v jeho podstrome tak sa pozrie, či nenastala úprava v koreni A . Riešenie sa síce spomalí, ale vďaka tomu že kontrolór je určený ako učebná pomôcka, vstupné formuly nebudú príliš veľké, a preto spomalenie je zanedbateľné.

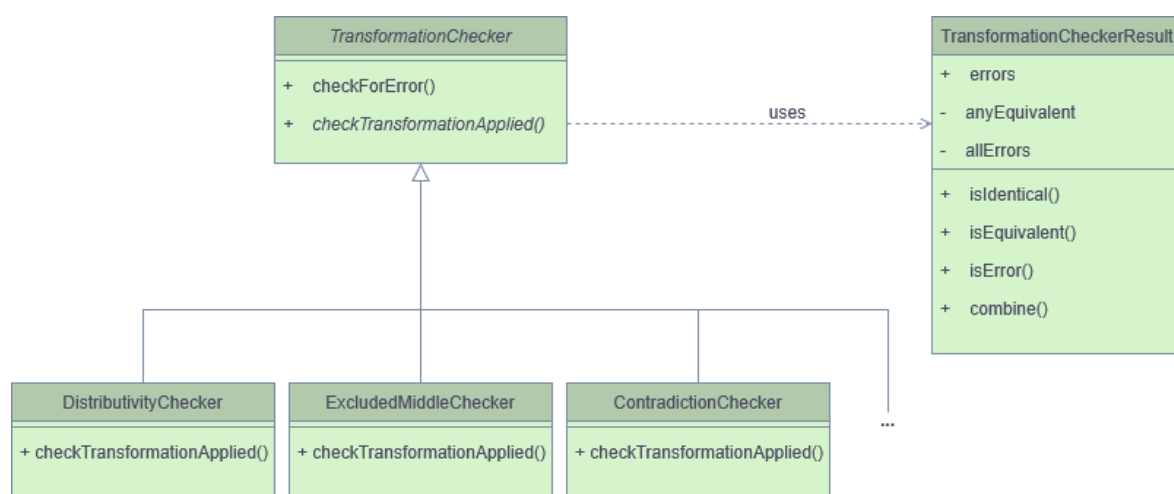
Výsledok kontroly bude reprezentovaný osobitnou triedou. Trieda výsledku bude obsahovať metódy na zistenie výsledku a tiež metódu na kombinovanie výsledkov. Možné hodnoty výsledku budú tri: identický, ekvivalentný a chybný. Identický výsledok znamená, že podstromy sú identické. Ekvivalentný výsledok je vrátený, keď v podstrome nastala aspoň jedna ekvivalentná úprava a všetky úpravy sú bezchybné. Chybný výsledok znamená, že v podstrome nastala buď chybná úprava, alebo iná chyba. Navyše obsahuje chybnú hlášku, ktorá sa neskôr zobrazí používateľovi. Rôzne ekvivalentné úpravy budeme reprezentovať osobitnými triedami pre každú úpravu. Triedy budú odvodené z abstraktnej triedy, ktorá bude obsahovať spoločnú časť algoritmu kontroly. Triedy sa budú líšiť v časti, ktorá usúdi či na konkrétnom mieste formuly nastala ekvivalentná úprava. Diagram tried je zobrazený v obrázku 3.1. Pseudokód spoločnej rekurzívnej časti kontroly je nasledovný:

```

checkForError(orig , trans) {
    if (sameFunctor(orig , trans)) {
        result = checkChildren(orig , trans)
        if (result.isEquivalentOrIdentical()) {
            return result
        }
    }
    return checkTransformationApplied(orig , trans)
}

```

Metódy `sameFunctor` a `checkChildren` budú súčasťou abstraktnej triedy. `sameFunctor` zistí či korene stromov sú rovnaké (či sú obidva rovnaké spojky, rovnaké quantifikátory s rovnakými premennými atď.). Keďže stromové reprezentácie formúl sú n -árne metóda `checkChildren` zavolá rekurzívnu kontrolu koreňov všetkých podstromov a skombinuje ich výsledky. Metóda `checkTransformationApplied` je abstraktná, ktoré konkrétne triedy prepíšu. Ak ekvivalentná úprava nastala, tak bude potrebné zavolať rekurziu na iné podstromy než tie ktoré skontrolovala `checkChildren`. Z toho dôvodu metóda tiež zavolá rekurzívnu kontrolu všetkých potrebných podstromov.



Obr. 3.1: Diagram tried kontroly ekvivalentných úprav

Kapitola 4

Implementácia

Kapitola 5

Testovanie

Záver

...záver...

Literatúra

- [1] Online dokumentácia bootstrap. <https://getbootstrap.com/>.
- [2] Online dokumentácia css. <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [3] Online dokumentácia html. <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [4] Online dokumentácia react. <https://react.dev/>.
- [5] Online dokumentácia react-bootstrap. <https://react-bootstrap.netlify.app/>.
- [6] Online dokumentácia redux. <https://redux.js.org/>.
- [7] Online dokumentácia typescript. <https://www.typescriptlang.org/>.
- [8] Jozef Šiška Ján Kluka, Ján Mazák. Prednášky z matematiky (4) – logiky pre informatikov. Dostupné z <https://fmfi-uk-1-ain-412.github.io/lpi/prednasky/poznamky-z-prednasok.pdf>.